MODULE 09 · CLASS SLIDES

# Tech 101 and System Design for PMs

Varun Kumar · SkillNavora

Prepared for Skillnavora · Learn. Practise. Implement. Grow.

# Tech for Product Managers

## Session 1

Understand how digital products work behind the screen.




A strong PM does not need to write code. A strong PM needs to understand how systems behave, where complexity hides, and how product decisions connect to technology.

1. **User Action** — Search for a product
2. **App / Frontend** — Collects input and shows results
3. **API / Gateway** — Validates and routes the request
4. **Services** — Business logic runs here
5. **Data** — Stored and organized
6. **Third-party** — External APIs & integrations
7. **Response** — Data returns to the user instantly

# Why PMs Need Tech Understanding

PMs do not need to become engineers. But they need to understand how products are built.

1

### Write Better PRDs

Define clearer requirements, edge cases, and constraints.

2

### Prioritize Better

Judge effort, dependencies, and scope more realistically.

3

### Work With Engineers

Ask sharper questions and reduce ambiguity.

4

### Debug Product Issues

Understand where the product may be breaking.

**Tech helps PMs connect user needs, business goals, and system reality.**



# The 4 Levels of Tech Understanding

Not every PM needs to code. But every PM should know enough tech to make better product decisions.

1

### Understand What Technology Enables

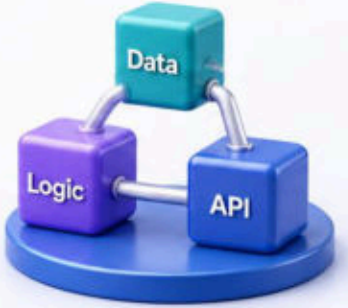
Spot product opportunities



2

### Understand the Building Blocks

Discuss feasibility with engineers




3

### Understand Tradeoffs and Design Choices

Make stronger scope decisions



4

### Build Hands-On Prototypes or Analyze Data

Validate ideas faster



**Most PMs should aim for Level 2.** Strong PMs gradually build Level 3.

# How We Will Learn Tech Today

A practical path from foundations to product decisions.

## 1 Internet, Web, and Digital Product Systems

How requests move across the web.




## 2 Frontend, Backend, APIs, and Databases

The core building blocks behind a feature.




## 3 Compute, Memory, Cloud, and Scale

Why performance, cost, and growth matter.



## 4 Product Flows and Hidden Complexity

What sits behind simple buttons like OTP, search, payments, and referrals.




## 5 Hands-On Case Study

Apply the concepts to a real product flow.




**We start with the screen, then uncover the system behind it.**

# Every Product Action Creates a System Journey

What the user sees is only one part of the story.

1. **User Action** — User taps a button
2. **Frontend** — Captures input and shows feedback
3. **API** — Sends the request
4. **Backend** — Runs business logic
5. **Database** — Stores and fetches data
6. **Third-Party** — Calls external services if needed
7. **Response** — Returns a result to the user




Strong PMs learn to see the system behind the screen.

# OTP Login: A Familiar Product Moment

We will use OTP login to understand how a simple product action triggers an entire system.

- Easy to relate to
- Includes frontend, API, backend, database, and SMS provider
- Has clear failure states
- Shows real PM decisions

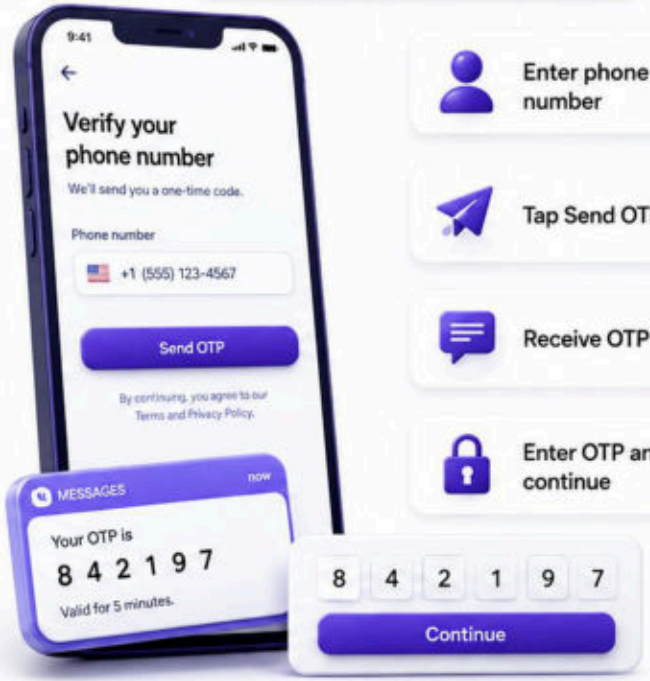



A simple login flow is a useful window into product systems.

# What the User Sees vs What the System Does

A PM needs to understand both layers.

## What the User Sees



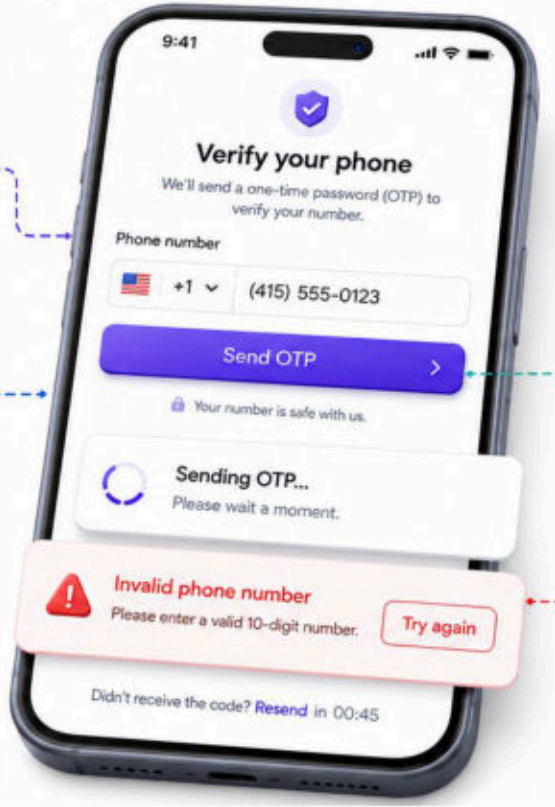

- Enter phone number
- Tap Send OTP
- Receive OTP
- Enter OTP and continue

## What the System Does

- Validate number format → +1 (555) 123-4567
- Check rate limits → 0 / 5 attempts remaining → OK
- Create OTP request
- Store attempt
- Trigger SMS service
- Prepare verification flow

# Frontend: What Happens on the Screen

The frontend handles what the user sees, enters, and feels during the flow.




### Input

Collects the phone number

### Validation

Checks format before sending

### PM Questions

- What should the CTA say?
- When should resend appear?
- What should the user see if OTP is delayed?

### Loading State

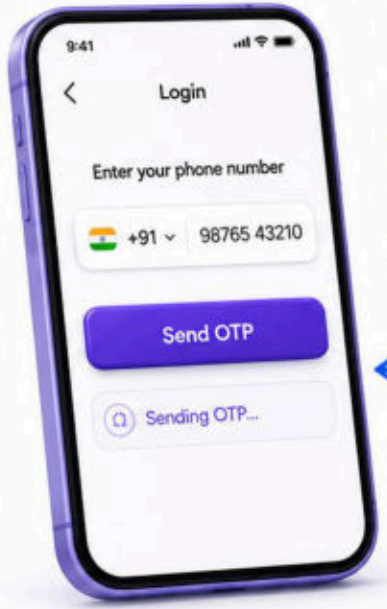
Shows progress while OTP is requested

### Error Handling

Handles invalid numbers, network issues, and retry states

# API: How ‘Send OTP’ Becomes a Request

The app sends a structured request and expects a structured response.




**Request**

```
{
  phone: "+91 98765 43210",
  deviceId: "a1b2c3d4"
}
```

**Response**

```
{
  requestId: "req_7f8a1c2d",
  status: "sent",
  resendAfter: 30
}
```




### Send OTP API

**Request:** Phone number, country code, device ID

**Response:** Request ID, status, resend timer

### Verify OTP API

**Request:** Phone number, request ID, OTP code

**Response:** Verified or failed, auth token, error reason

## Why PMs Should Care

### Reliability

Fewer failures, better user trust.

### Latency

Faster responses = better experience.

### Retry behavior

Resilient flows reduce

# Backend: The Logic Behind OTP Login

The backend applies rules, generates OTPs, and decides whether the login can proceed.

### Validate Request

Check input, device, and request integrity.

### Generate OTP

Create a secure, random OTP and deliver it.

### Apply Rate Limits

Prevent abuse with smart throttling and attempt controls.




**Business logic** runs here.

### Check Existing User

Verify if the user exists and fetch profile status.

### Create Session After Success

Establish a secure session and issue tokens.

### Log Events for Security

Record attempts, results, and metadata for audit and monitoring.

**What feels simple to the user often depends on multiple backend rules.**

# Database: What the System Stores

The database remembers users, OTP attempts, verification status, and session data.

### User Record

Phone number, country code, signup status

### OTP Attempt Log

Request ID, created time, expiry, retry count

### Verification State

Sent, verified, failed, expired




### Session Data

Auth token or session after success

### Audit Trail

Device metadata and security logs

**PMs should know what data gets written, updated, and checked in every flow.**

# Third-Party Services: Sending the OTP

A simple login flow often depends on an external SMS provider.

### Delivery Request

The app passes the OTP to the provider

### Routing

The provider sends it through telecom networks

### Delivery Status

Sent, delayed, or failed updates come back

### Fallbacks

Retry, resend, or alternate channel support




### Common Failure States

- Delayed SMS
- Country restrictions
- Delivery failure
- Resend abuse

Vendor **reliability**, **latency**, and **fallback design** shape the login experience

# Putting the OTP Flow Together

One simple product action touches multiple layers of the system.




User Enters Number → Frontend Validates → API Sends Request → Backend Generates OTP → Database Stores Attempt → SMS Provider Delivers Code → User Verifies and Continues

**Strong PMs learn to see the full system, not just the screen.**

# Client and Server: Who Does What?

The client handles the interface. The server handles logic, rules, and data.




**Client / App**

What the user sees and interacts with.

**Request**

Send data / action to the server

**Response**

Return data / result from the server



**Server / Backend**

Handles logic, security, and data management.

| | |
|---|---|
| **Input Fields** | Collect user data |
| **Local Validation** | Check format before sending |
| **Loading States** | Show progress and feedback |
| **Error Messages** | Help the user recover |

| | |
|---|---|
| **OTP Generation** | Create the code securely |
| **Auth Rules** | Validate and verify the request |
| **Rate Limits** | Prevent abuse and spam |
| **Data Access** | Read and update stored information |

**A PM should know which decisions belong on the client and which belong on the server.**

# Internet vs Web: The Base Layer

The internet is the network. The **web** is one service built on top of it.

## Internet

A global network of devices, routers, cables, and servers.



## Web

Websites and web apps accessed through browsers and HTTP.




## PM Relevance

- Performance issues may come from the network, not only the app.
- Web products still depend on servers and infrastructure.
- Understanding the base layer helps PMs ask better questions.

**When a product feels slow, the issue may be in the app, server, or network.**

# Request And Response: How Digital Products Talk

Most digital experiences begin when the app sends a request and receives a response.

1




**User Action**

User taps search or places an order.

2



**Frontend Sends Request**

The app prepares and sends the request.

3




**API/Server Receives It**

The server receives and validates the request.

4



**Backend Processes Logic**

Business logic runs, applying rules and transformations.

5



**Database Fetches/Updates Data**

Data is read from or written to the database.

6



**Response Returns To User**

The response is sent back and displayed on the app.

**Example: Search For Running Shoes**

- User enters query
- Backend finds matching products
- Database fetches product data
- Results appear on screen

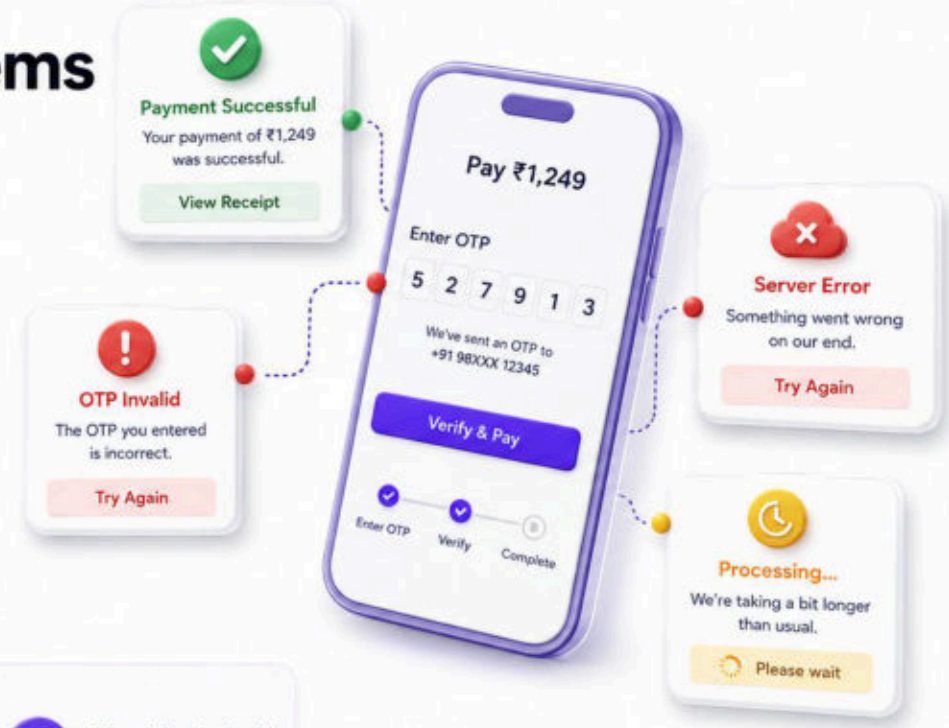
When a product feels slow, a PM should ask where the request is getting stuck.

# API Errors Become User Experience Problems

What looks like a technical error often becomes a trust and conversion problem for users.

## Common Status Codes

| | | | |
|---|---|---|---|
| ✓ | 200 | Success | Show result |
| ! | 400 | Bad Request | Ask user to fix input |
| 🔒 | 401 | Unauthorized | Ask user to log in |
| 🔍 | 404 | Not Found | Show unavailable state |
| ☰ | 500 | Server Error | Retry or fallback |
| 🕒 | 503 | Service Unavailable | Temporary issue |




## PM Questions

- What should the error message say?
- Should the system retry automatically?
- What fallback should the user see?

Great PMs do not just define the happy path. They define how failure should f

# Databases: Product Memory Across Features

A database stores the product state that makes user experiences possible.

## Why PMs Should Care

- Search and filters
- History and audit trails
- Personalization
- Analytics and reporting



### Food Delivery

- users
- restaurants
- menu
- orders
- payments



### Social App

- profiles
- posts
- comments
- likes



### Edtech

- learners
- courses
- sessions
- assignments





### Fintech

- users
- cards
- transactions
- rewards

## Typical Questions

- What data gets saved?
- What gets updated?
- What should be recoverable?

# SQL Vs NoSQL: Different Ways To Store Data

PMs do not choose databases alone, but they should understand why data structure affects the product.



## 1 SQL

Structured tables and relationships.

Best for orders, payments, CRM.




## 2 Document DB

Flexible JSON-like records.

Best for profiles, content, settings.



## 3 Graph DB

Relationship-heavy data.

Best for social networks and recommendations.




## 4 Key-Value DB

Fast lookups with simple keys.

Best for sessions, OTPs, cache.

## PM Implications

**Reporting**
Ease of analytics and insights.

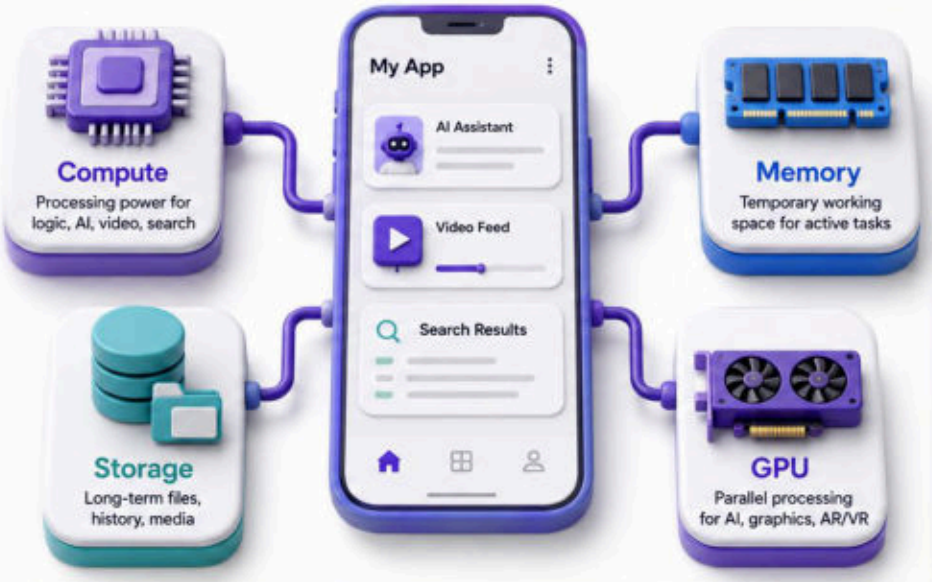
**Flexibility**
Adapt to change without heavy cost.

**Performance**
Meet response time and latency goals.

**Scalability**
Handle growth in data and traffic efficiently.

# Compute And Memory: Why Products Become Slow

Performance issues often come from the resources a feature consumes behind the scenes.




### What PMs Hear

- This feature is heavy
- This may slow the app
- This needs more infrastructure

### What It Usually Means

- High compute load
- Too much memory use
- Network or media processing cost

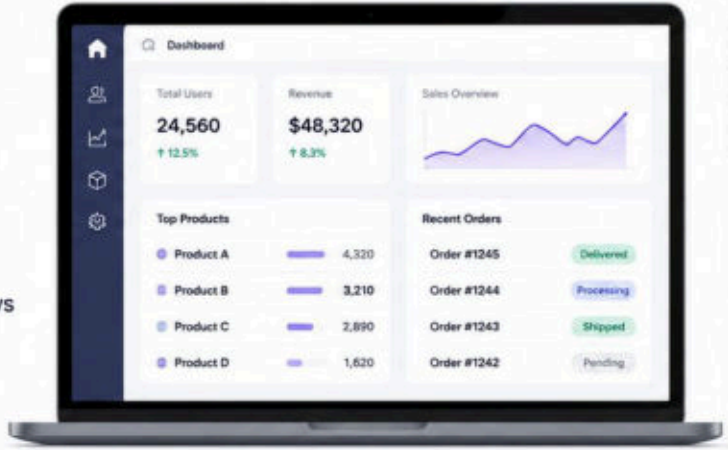
**A strong PM connects performance complaints to the underlying technical constraint.**

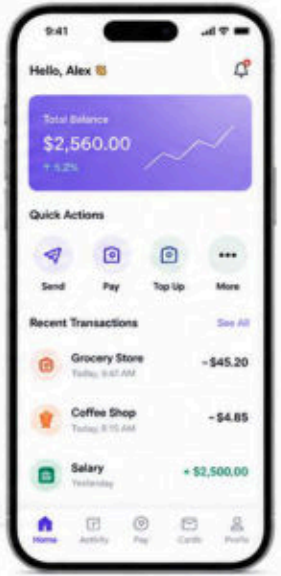
# Web vs Mobile:
# Same Logic, Different Constraints

The product logic is similar.
The environment changes everything.

## Web

- More screen space
- Always connected
- Powerful browsers
- Great for complex flows
- Ideal for deep work




## Mobile

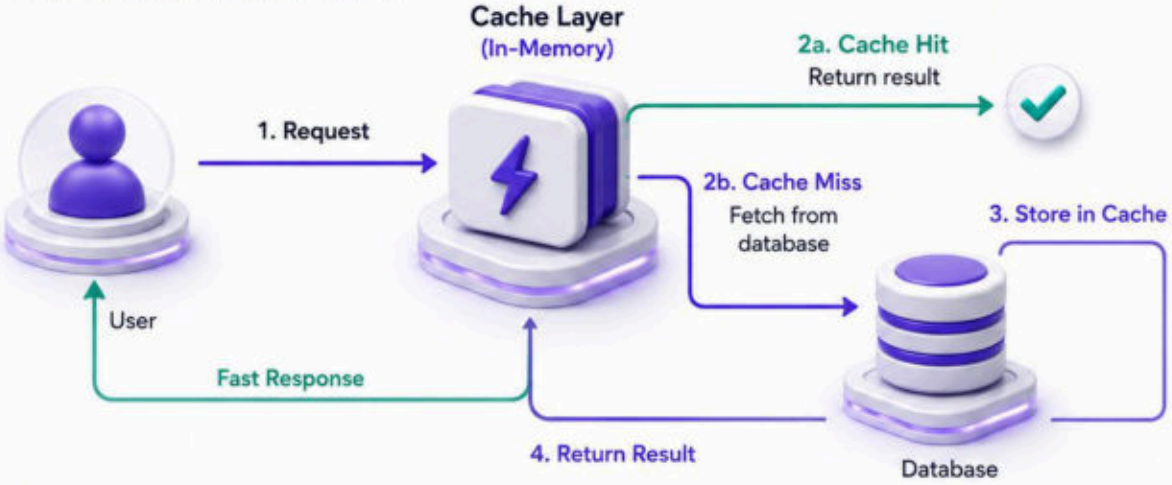
- Limited screen
- Network can be unstable
- Battery and performance constraints
- Great for on-the-go, simple actions
- Optimized for touch and quick interactions

**PM takeaway:** Design and prioritize with platform realities in mind.
What works on web may not work on mobile—and vice versa.

Same goal.
Different constraints

# Caching: Speed Through Smart Reuse

Caching stores frequently used data closer to the user so the system can respond faster.




## Why It Matters

**Faster response times**
Serve data from memory instead of slower sources.

**Reduces load on databases**
Fewer database calls mean less strain.

**Lower infrastructure costs**
Use fewer resources to deliver better performance.

**PM Takeaway:** Caching improves speed, but stale data and invalidation are trade-offs.

# CDN: Deliver Content From Closer Locations

A CDN stores static content on servers around the world to reduce latency and improve performance.




**Lower Latency**
Content is delivered from the server closest to the user.

**Better Global Performance**
Faster load times, higher reliability, and improved user experience.

## What Can Be Served Via CDN?

- Images
- Videos
- Stylesheets (CSS)
- JavaScript files
- Fonts
- Downloads

**PM Takeaway:** CDNs improve global performance and user experience at scale.

# Asynchronous Processing: Handle Long Tasks Without Blocking Users

Some tasks take time. Don't keep the user waiting. Process them in the background.

## Fast User Flow

Keeps the user moving.

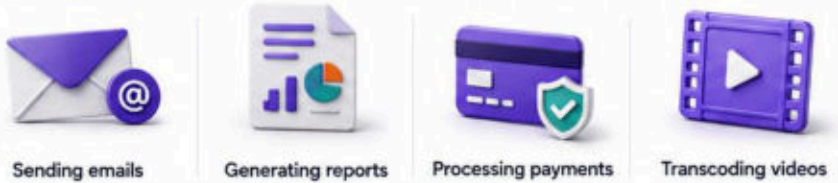



## Background Async Flow

Handles the work off the user's path.




## Examples




## Benefits

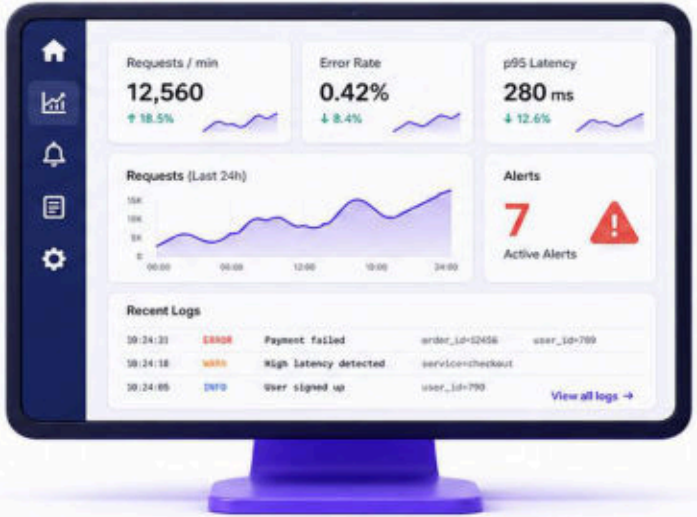
- **Better user experience**
  Users don't wait for long tasks to finish.
- **Improves system reliability**
  Failures are isolated and easier to recover from.
- **Handles more load efficiently**
  Scales by adding workers, not blocking users.

# Logging & Monitoring: See What's Happening, Fix What Matters

You can't improve what you can't see. Logs show what happened.
Monitoring shows how the system is doing.

## Logging (What Happened)

- Records events, errors, and important info
- Helps debugging and root cause analysis
- Stored for search and investigation
- **Example:** Error stack trace, user signup




## Monitoring (How It's Doing)

- Tracks system health in real time
- Alerts on issues or anomalies
- Helps proactive decision making
- **Example:** High latency alert, server down

**PM Takeaway:** Good observability reduces downtime, faster recovery, and better user experience.



# Case Study: Break Down CRED Onboarding

Map the user journey to the systems that power it.

## 1 Enter Phone Number




User enters mobile number to begin.

## 2 Send OTP




System triggers OTP through SMS.

## 3 Verify OTP




User confirms identity with OTP.

## 4 Enter Details



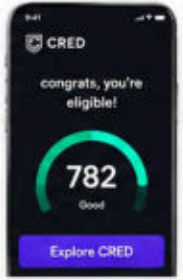

Collect profile and identifiers.

## 5 Fetch Credit Score




Call bureau API and fetch score.

## 6 Show Eligibility




Display eligibility and next steps.

## Underlying Systems And Dependencies



**Frontend Validation**

Input format and basic checks



**OTP API**

SMS gateway, rate limits, retries



**Backend Authentication**

Verify OTP and issue session



**Profile Database**

Store profile, consent, identifiers



**Third-Party Credit Bureau**

Fetch score using secure APIs



**Loading And Fallback States**

Timeouts, retries, graceful messa

## PM Questions

- What if OTP is delayed?
- What if the bureau API fails?
- What consent is needed?
- What data should be stored?
- What should users see while waiting?

# Web Application Architecture

Web and mobile products may look different, but they often rely on similar underlying systems.




A PM should understand how each layer contributes to the user experience

# Session 1 Recap

You should now be able to explain the main building blocks behind a digital product.



### Frontend

The user interface runs in the browser and sends requests.



### Backend

The server-side logic processes requests and contains rules.






### API

APIs define how systems communicate through structured requests and responses.



### Database

Databases store and organize data so it can be retrieved quickly and reliably.



### Compute And Memory

Servers use compute and memory to run code and handle scale and load.



**A PM does not need to write the code, but should understand the system well enough to ask the right questions.**

# Using Tech Thinking As A PM

From **technical concepts** to **product decisions.**




Session 2 focuses on how PMs use **technical understanding** to make **better product decisions.**

# What We'll Cover Today

From technical understanding to sharper PM decisions.

1



## From Tech Concepts to PM Decisions

Learn how PMs translate technical understanding into product trade-offs and prioritization.

2



## Thinking in Systems: Product Flow & Failure Scenarios

Analyze real product journeys (e.g., payment/order flows) and identify failure points, edge cases, and dependencies.

3




## Writing Better Product Requirements (FR vs NFR)

Understand functional vs non-functional requirements and how to write technically grounded PRDs.

4



## PM Decision Frameworks: Tradeoffs, Metrics & Build vs Buy

Evaluate scalability, latency, reliability, vendor choices, and business impact.

5



## Hands-On Product Inspection & Case Study

Apply concepts through product flow analysis, debugging, and practical PM exercises.

# Functional Vs Non-Functional Requirements

FR defines what the system should do. NFR defines how well it should work.



## Functional Requirements

- User can upload a file
- User can receive a notification
- User can send a message
- User can view order history

VS



## Non-Functional Requirements

- Upload completes within 10 seconds for files under 25 MB
- Payment confirmation arrives within 10 seconds
- Messages are delivered with low latency
- Order history loads in under 2 seconds

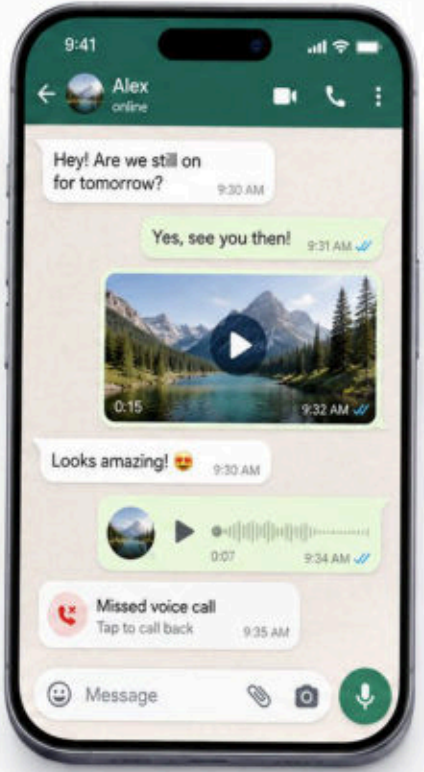
## Why It Matters

- Reduces ambiguity
- Aligns engineering and QA
- Improves user experience

**PM Takeaway:** Great PMs define both what the feature does and how reliably it should behave.

# WhatsApp Example: FR And NFR

One feature can have clear functional expectations and equally important quality expectations.




## Functional Requirements

- Send text messages
- Share images and videos
- Show delivery and read status
- Receive push notifications
- Access past chat history

## Non-Functional Requirements

- Messages should deliver with low latency
- Media should upload on unstable networks
- Status updates should be reliable
- Notifications should arrive quickly
- Chats should load fast

**PM Takeaway:** A feature is not complete until you define how well it should work in real-world conditions.

# Bad Requirement Vs Good Requirement

Good requirements reduce ambiguity and improve execution.

| | Weak Requirement | Stronger Requirement |
|---|---|---|
| 1 | "App should be fast" | Search results should load within 2 seconds for 95% of searches |
| 2 | "User can upload file" | User can upload PDF files up to 25 MB with progress, retry, and error state |
| 3 | "User gets notification" | User receives payment confirmation within 10 seconds after successful transaction |
| 4 | "System should be secure" | Sensitive payment data should not be stored unless required by compliance |

## Why Stronger Requirements Help

- Clearer engineering scope
- Better QA coverage
- Fewer edge-case surprises

**PM Takeaway:** Specific requirements create better products and faster alignment across teams.

# Product Tradeoffs

Every technical choice creates a product tradeoff.

### 1. Speed Vs Accuracy

Should the AI answer faster or more correctly?

### 2. Build Vs Buy

Should we build this capability or use a vendor?

### 3. Real-Time Vs Delayed

Does the user really need instant updates?



### 4. Cost Vs Performance

Should we spend more infrastructure to improve speed?

### 5. Security Vs Convenience

Should login be stricter or smoother?

### 6. MVP Vs Scale-Ready

Should we build fast or future-proof?

### How PMs Use This

- Frame decisions clearly
- Expose hidden costs
- Align tradeoffs with user value

**PM Takeaway:** Great product decisions come from clear tradeoff thinking, not perf

# You Don't Need To Code, But You Need To Think In Systems

Great PMs don't write code.
They understand how things work together and why.

## What This Means

- Ask better questions
- Understand impact
- Make confident decisions
- Collaborate with engineers
- Build products that scale



# Build Vs Buy

Should we build this capability ourselves or use a vendor?




## Build

- More control
- Custom experience
- Strategic differentiation
- High engineering effort
- Slower launch

## Buy

- Faster launch
- Proven vendor stack
- Ongoing vendor cost
- Dependency risk
- Possible lock-in

**Example capabilities**

Payments | Maps | Chat | KYC | Analytics

**Decision criteria**

- Is this core to differentiation?
- How fast must we launch?
- What is the cost at scale?
- What happens if the vendor fails?
- Can we switch later?

**Build what makes you different. Buy what helps you move faster**

# Monolith Architecture

One application handles many product functions inside a single codebase.

## What It Is

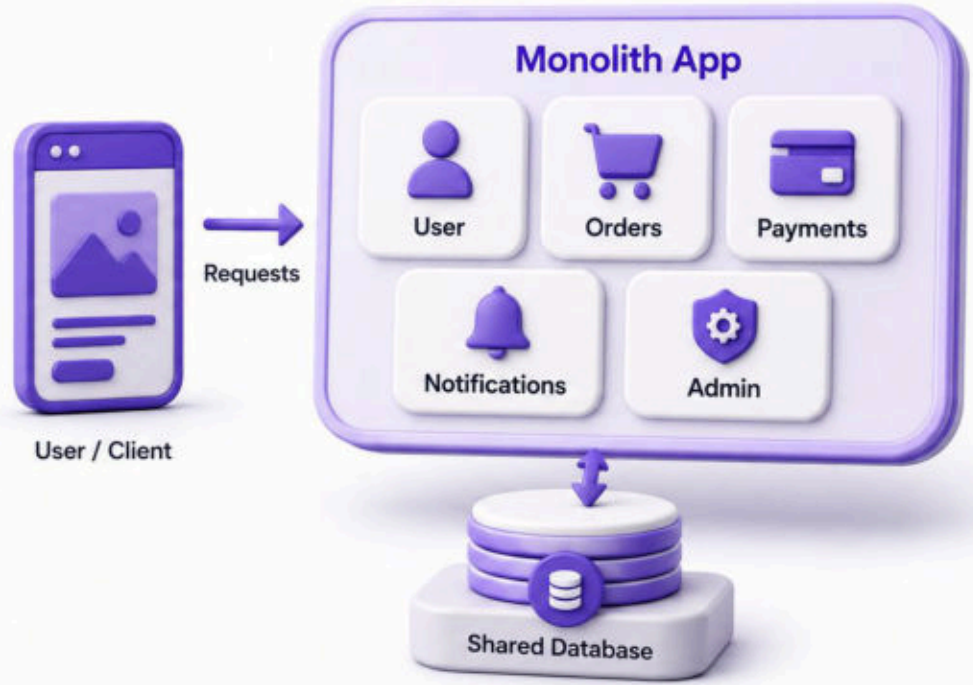
A monolith keeps core product logic in one deployable application.

## Why Teams Start Here

- Simpler to build early
- Easier local development
- Single deployment pipeline

## PM Implications

- Fast to launch MVPs
- Tighter coupling across features
- Changes in one area can affect the whole app




**Best for early-stage products when speed and simplicity matter more than service-level independence.**

# Microservices Architecture

The product is split into smaller services that communicate through APIs.

## What It Is

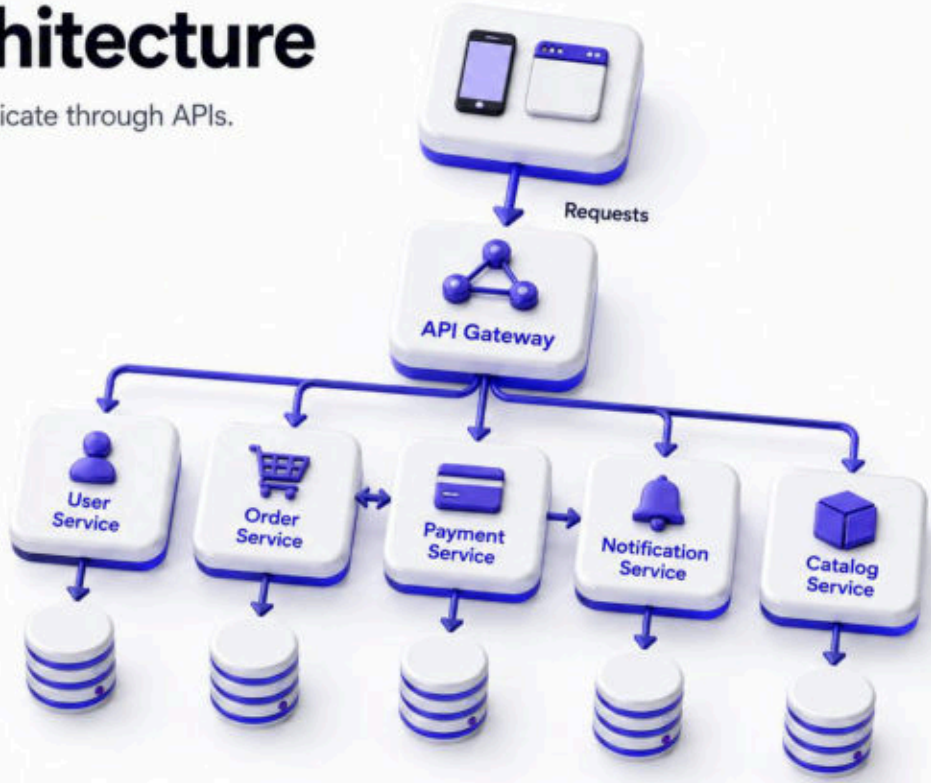
Each service owns a specific capability and can evolve more independently.

## Why Teams Use It

- Independent service ownership
- Scale only the busy parts
- Separate deployments by domain

## PM Implications

- Better fit for growing products
- More coordination across teams
- More failure points and observability needs

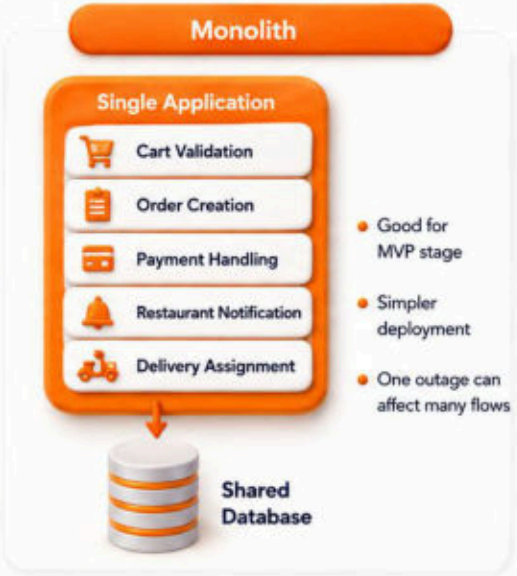


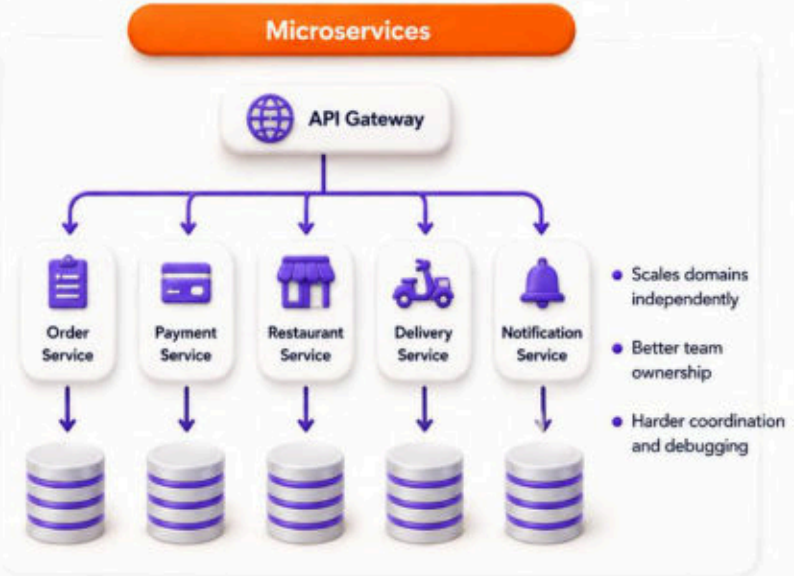

**Best for products with growing scale, multiple teams, and domain-level complexity.**

# Monolith vs Microservices in Practice

A Swiggy order flow shows when the difference starts to matter.




| | Monolith | Microservices |
|---|---|---|
| Best When | Early-stage product | Growing multi-team product |
| Main Strength | Simplicity | Flexibility and scale |
| Main Risk | Tight coupling | Operational complexity |

# Sprint Planning: Choosing the Right Work

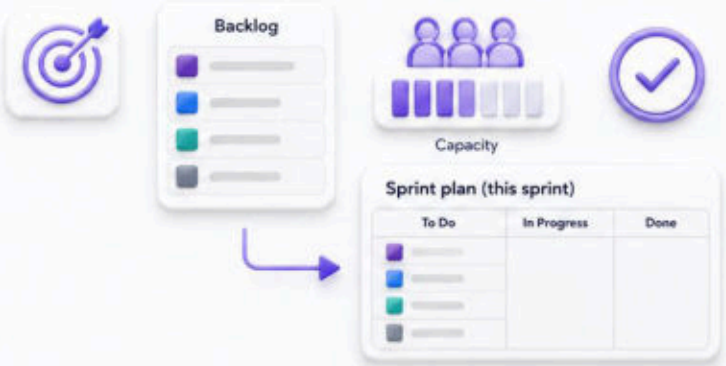
Sprint planning helps the team decide **what to build next** and **what can realistically be completed** in the sprint.

## Inputs

- Product goal
- Prioritised backlog
- Team capacity
- Dependencies

## What happens in sprint planning?

1. Pick the sprint goal
2. Review the highest-priority items
3. Check effort and team capacity
4. Commit to the work for the sprint




## Output

**Sprint Plan**

| | |
|---|---|
| US-101 | As a user, I can reset my password |
| US-102 | As a user, I can update my profile |
| US-103 | As a user, I can view my notifications |
| US-104 | As a user, I can log out securely |

Committed

A clear sprint goal and a realistic set of stories for the sprint.

## PM relevance

Sprint planning is not about adding more work.
It is about **choosing the right work the team can finish well.**

# Backlog Grooming: Making Work Ready

Backlog grooming helps the team turn rough ideas into clear, buildable work before sprint planning.

## What backlog grooming fixes

- Vague ideas
- Stories that are too large
- Missing acceptance criteria
- Hidden dependencies

## How a PM grooms the backlog

1. Clarify the user problem
2. Rewrite as a clear user story
3. Add acceptance criteria
4. Split large work into smaller stories

**Before**

Improve onboarding

**After**

**User story:** As a new user, I want guided onboarding so I can set up faster.

**Acceptance criteria:**
- 3 setup steps
- progress visible
- can skip and return

**Priority:** High

**Size:** 5 pts

## Output

**Ready backlog item**

- User story
- Acceptance criteria
- Priority
- Size
- Dependencies

A groomed backlog gives the team work that is clear, scoped, and ready for planning.

## PM relevance

Good grooming reduces confusion, improves handoff quality, and makes sprint planning faster.

# Estimation: Understanding Effort and Uncertainty

Estimation helps the team discuss **effort**, **complexity**, and **risk** before committing to work.

## What teams estimate

- Complexity of the work
- Unknowns and risk
- Dependencies and coordination
- Testing and validation effort

## A simple way to estimate

1. Read the story
2. Discuss unknowns
3. Compare with past work
4. Agree on a relative estimate

**1–2 points**



Small, clear, quick work

**3–5 points**



Medium, some unknowns

**8+ points**



Large, high uncertainty, may need splitting

## Example estimates

| | |
|---|---|
| Reset password flow | 3 pts |
| Add social login | 5 pts |
| Improve onboarding progress bar | 3 pts |
| Build referral dashboard | 8 pts |

Large items are a signal to split or clarify, not a reason to guess harder.

## PM relevance

Estimation helps PMs understand delivery trade-offs and avoid over-committing the team.

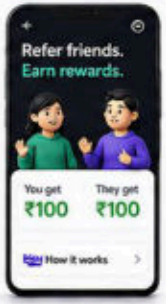
# Case Study: CRED Referral Flow

How a refer-a-friend journey maps to systems behind the scenes.

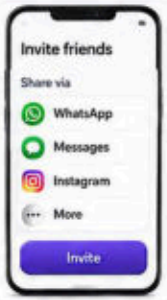
## User Journey

What the user sees and does.

### Open Referral Page




### Tap Invite




### Generate Link Or Code




### Friend Signs Up



Welcome!

Account created successfully.

### Validate Eligibility

- KYC verified
- 1st bill paid
- Within 30 days

Eligible

### Issue Reward



Reward credited!

₹100 added to your balance.

### Run Abuse Checks



Checks complete

No policy violations detected.

## Systems

What happens behind the scenes.

### Frontend Share Flow

Handles deep links, pre-fills invite context, and share channels.

### Referral API

Creates referral records and returns link/code.

### Attribution Engine

Maps friend sign-ups to inviter using link, code, or device graph.

### Reward Rules

Evaluates eligibility rules and reward configuration.

### User Database

Stores users, KYC, transactions, and referral relationships.

### Fraud Detection

Detects abuse patterns, duplicate behavior, and policy violations.

### Notifications

Sends updates on status, rewards, and reminders.

## PM Questions

- Who gets credit if multiple users invite the same friend?
- When should reward be issued?
- How do we prevent fake referrals?
- What should th… see while rewa… pending?

# Referral Flow: PM Decisions

The referral system is simple on the surface, but the product decisions are not.

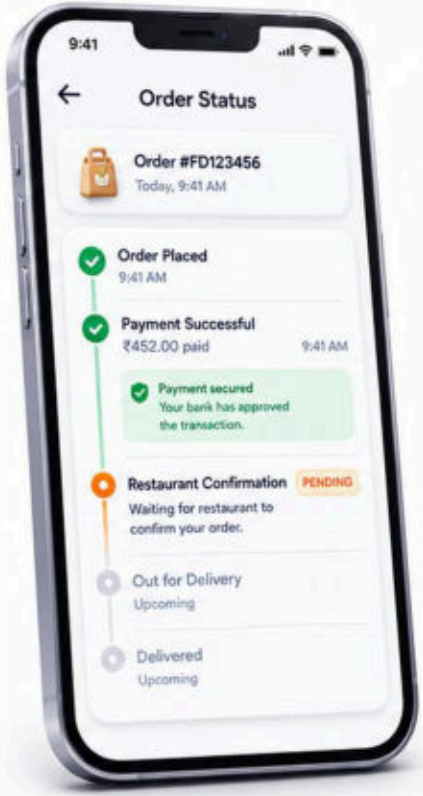



**Good decisions drive outcomes**

Trust | Lower Fraud | Better Conversion | Clearer Support

# Case Study: Payment Done, Restaurant Did Not Receive Order

A food delivery failure where payment succeeds but restaurant confirmation breaks.




1. User Places Order — SUCCESS
2. Backend Creates Order — SUCCESS
3. Payment Captured — SUCCESS
4. Webhook / Event — FAILED
   - **Webhook Delayed**: Restaurant not notified in time.
5. Restaurant Notification — FAILED
   - **Restaurant System Unavailable**: Order not received by restaurant POS.
6. Delivery Allocation — BLOCKED
   - **No Allocation**: Cannot assign rider as order is not confirmed.
7. User Status — INCORRECT
   - **Wrong Status**: User sees "Preparing" or "On the way", incorrectly.

## Possible Failure Points

- **Frontend request failure**: Order request doesn't reach the server.
- **Order not saved**: Order creation fails or rolls back.
- **Webhook delayed**: Event not delivered or times out.
- **Restaurant system unavailable**: POS offline or rejects the incoming order.
- **No delivery partner**
- **Wrong user status shown**

# Food Delivery Failure: PM Response

When the system fails, the PM must define the fallback experience.

## What Should The PM Define?



### 1 User Status

Show a transparent message such as "Payment received, confirming order".



### 2 Auto-Retry

Retry restaurant notification or allocation for a limited period (e.g., 5 minutes).



### 3 Refund

Trigger a full refund if confirmation fails within a defined time (e.g., 15 min).






**Fallback Activated**

Define the experience the user receives next.





### 4 Support

Auto-create support tickets for high-risk or high-value orders.



### 5 Communication

Notify the user proactively with updates before they need to contact support.



### 6 Metrics

Track payment success but order failure rate, resolution time, and refund rate.



**Good fallback design protects trust even when the system breaks.**

# AI Summary Tradeoffs

AI features force product choices across speed, quality, trust, privacy, and control.




**PM Takeaway:** AI features are not just model decisions. They are product tradeoff decisions

# AI Summary Requirements

Translate the AI idea into clear product and system expectations.

### Functional
Generate a meeting summary from the transcript and extract action items.

### Latency
Summary should be available within 2 minutes after the meeting ends.

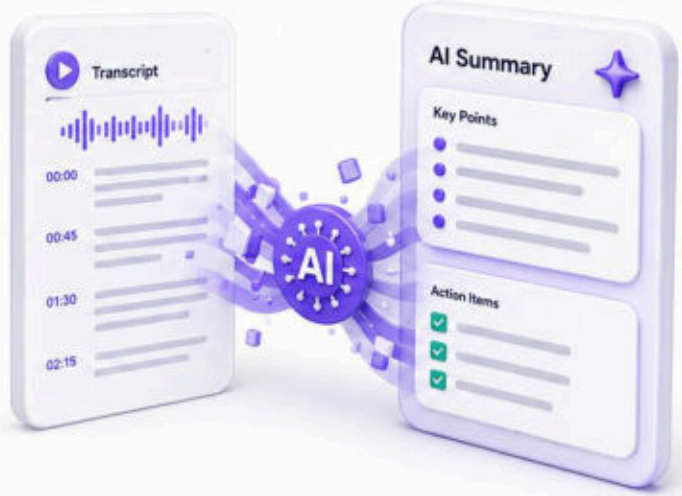
### Accuracy
Users should be able to review and edit the summary before sharing.

### Privacy
Transcript should not be shared or stored without clear consent.

### Fallback
If AI fails, show the transcript and mark summary as unavailable.

### Evaluation
Track edit rate, share rate, and helpfulness rating.

### Risk
Do not auto-assign incorrect action items without user review.




**PM Takeaway:** Good AI requirements define behavior, quality, risk, and fallback — not just output.

# Latency

How long can the user wait before the experience starts to break?

Instant → Delayed

| Typing Message | Search Results | Payment Confirmation | Video Upload | AI Image Generation | Monthly Report |
|---|---|---|---|---|---|
| Instant | 1–2 seconds | A few seconds | Longer is okay with progress | Longer is okay with queue and status | Delayed is okay |



**PM Question**

What should the user see while waiting?

**PM Takeaway:** Latency is not just speed. It is expectation design.

# Scale

What breaks when usage, geography, or complexity grows?




## PM Relevance

- Ask what assumptions the system is built for
- Identify the first thing that breaks
- Plan for spikes, not just averages

**PM Takeaway:** Scale problems usually appear before users describe them clearly.

# Reliability

When systems fail, the product still needs a trustworthy fallback experience.

| | |
|---|---|
| Payment Gateway Down | Offer an alternate payment method |
| SMS OTP Delayed | Allow resend and voice OTP |
| Delivery Tracking Unavailable | Show the last known status |
| AI Summary Failed | Show transcript only |
| Search Service Down | Show cached or recommended results |
| Vendor API Failed | Show a graceful fallback state |



## Why It Matters

- Protects trust
- Reduces support load
- Keeps users moving

**PM Takeaway:** Reliability is not about preventing every failure. It is about designing smart fallbacks.

# Security And Privacy

Security and privacy are product decisions too.

**1. Authentication**
Who is this user?

**2. Authorization**
What can this user access?

**3. Consent**
Has the user agreed?

**4. Data Minimization**
Do we need to collect this data?

**5. Encryption**
Is sensitive data protected?

**6. Abuse Prevention**
Can this be misused?

**7. Compliance**
Are there legal constraints?




**PM Takeaway:** Security and privacy choices shape trust, risk, and product experience.

# Cost: Tech Cost Becomes Product Cost

Every technical decision has a cost.
Smart PMs see the full picture.

## Types Of Cost To Consider

**Development Cost**
Engineering time, tools, and complexity.

**Infrastructure Cost**
Servers, databases, APIs, third-party services.

**Operational Cost**
Monitoring, support, incident management.

**Maintenance Cost**
Tech debt, upgrades, and ongoing fixes.

**Opportunity Cost**
What we could build instead.

## Small Tech Choices, Big Product Impact

| Tech Decision | Hidden Cost | Product Impact |
| --- | --- | --- |
| Real-time processing | Higher infra cost and complexity | May improve UX, but can hurt margins |
| Third-party service | Recurring fees + dependency risk | Faster launch, but less control |
| Storing more data | Storage, compliance, and privacy cost | Better insights, but higher long-term cost |

**PM Mindset:** Balance value for users with value for the business.

# Tools PMs Can Use

PMs do not need to master every tool. They should know what each tool helps them answer.




## Browser DevTools

Inspect API calls, errors, and loading




## API Client

Test requests and responses




## API Docs

Understand endpoints and payloads



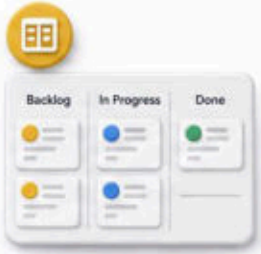
## Event Analytics

Analyze user behavior and funnels




## Error Monitoring

Understand crashes and performance




## Docs And Tracking

Read specs and follow scope

**PM Takeaway:** Use tools to ask better questions, not to replace engineering.

**Tech 101 Assignment**

**Project brief**

Imagine you are a product manager at an online education platform, tasked with designing a personalised learning recommendation system.

The goal is to help users discover relevant courses based on their interests, skill levels, and career goals, leading to increased engagement, completion rates, and overall satisfaction.

**Business Objectives:**

- Your system should aim to:
- Increase user engagement by 25%
- Improve course completion rates by 30%
- Enhance user satisfaction scores by 20%

Design for three user personas:

- Learners
- Instructors / Content Creators
- Platform Admins

Your assignment is divided into six structured steps. Follow the steps mentioned below to reach your goal.

**Submission guidelines**

- Your submission must be in .PDF file format only.
- Your submission must also include a video link (loom video or google drive link only).
- Make sure you are visible in the video along with your presentation, and the video must strictly not exceed 5 minutes
- Directly copying solutions from tools like ChatGPT is not advisable and would lead to your assignment being rejected by the mentor.

**Deliverables**

- PDF file
- External link

**STEP 3**

**Non-Functional Requirements**

List and explain 1–2 non-functional requirements in each of these categories:

- Performance
- Scalability
- Availability
- Compliance
- Security & Data Retention
- Monitoring & Logging
- Extensibility

**STEP 4**

**Core System Design**

Outline the main system components needed to deliver your user stories.

Cover:

- User interface
- Backend system
- Platform integration

You may present this using a block diagram or flowchart.

**STEP 5**

Trade-Off Analysis

Identify one major trade-off in building the system (e.g., personalization vs. performance or privacy vs. tracking).

Explain:

- What's in conflict
- How you'd prioritize
- The product decision you would make and why

**STEP 6**

- Assumptions & Future Planning
- List your key assumptions and outline possible improvements or extensions that could be explored in the future.

**STEP 7**

Final Deliverables

- A detailed PDF file with all six steps covered
- A Loom or Zoom video walkthrough (~5 minutes max) explaining:
- Your thought process and trade-offs
- System design logic and feature choices
- How your solution aligns with the business goals

# Final Recap

Tech for PMs is not about jargon. It is about product judgment.

### 1 See the System Behind the Screen




- Break features into layers
- Understand frontend, backend, API, and data

### 2 Ask Better Technical Questions



- Clarify complexity
- Find the real bottleneck

### 3 Write Clearer Requirements



- Define what the system should do
- Specify how well it should behave

### 4 Think Through Failures



- Map edge cases
- Plan fallbacks

### 5 Make Better Tradeoffs



- Balance speed, cost, reliability, and UX
- Choose intentionally

### 6 Collaborate With Engineering Better



- Speak in systems
- Align on constraints and scope